

Rental tracker

Now that you have an idea of how databases work and a better understanding of working in a shared repo, you'll build an app that needs to remember things between runs.

You bought a ladder once, used it twice, and now it's collecting dust under the stairs. Meanwhile, someone a few streets away is about to buy the exact same ladder for a one-time job. Why not rent your ladder out to them for a small price?

That works at first. But after a few rentals, things blur. You can't quite remember whether the ladder came back or which rental is overdue. As more items go out, keeping track of what's rented, to whom, and for how long stops being obvious and becomes a problem.

This topic focuses on building CLI applications that manage persistent data and evolve from single-user tools into more reliable multi-user systems.

In the first project, you'll build a personal rental tracker that lists items, shows your inventory, rents things out, and confirms returns. The focus is on the foundations: storing data in a database that survives between runs, designing its schema, and structuring the code in clear layers.

In the second project, you'll open the rental tracker up into a multi-user marketplace. The focus shifts to making it robust: handling bad input and errors cleanly, logging what the app does, and packaging it with Docker so it runs anywhere.

Holding all of this together as the app grows takes deliberate structure, which is where the layers come in.

Architecture

The code is split into five layers, each with one job.

- **Transport** - the part the user sees: the menus, the lists, the detail views, and the prompts for things like username, item fields, renter name, and duration.
- **Service** - the decision-maker: whether an item can be rented, what its end time is, finding or creating a renter, and allowing only valid status changes.
- **Repository** - the only part that talks to the database: every insert, lookup, and status update goes through here, so nothing above it writes SQL or touches SQLite.
- **Domain** - the building blocks the rest of the app passes around: the User, Item, and Rental classes.
- **Infrastructure** - the code that opens and manages the database connection (through a driver like `org.xerial:sqlite-jdbc`).

How you organise files and packages inside each layer is up to you.



Main menu

The app's main menu is where every action starts. Choosing an option opens that feature, and finishing it returns to the menu, until the user exits. For example:

```
=== Rental tracker ===  
1) List an item  
2) View my inventory  
3) Record a rental  
4) Confirm a return  
5) Exit  
>
```

What each option does:

- **List an item** - create a listing for one of your items so it can be rented.
- **View my inventory** - see your items and their status, and open one in its detail view to delist or relist it.
- **Record a rental** - rent one of your available items out to someone.
- **Confirm a return** - mark a rented item as returned.
- **Exit** - close the program cleanly: the database connection is closed and no process is left hanging.

User account

Before anything can be listed or rented, the app needs to know who you are.

On first launch, the app asks for a username and stores it in SQLite as your account. In this project that account is the single owner behind every listing. On every run after that, the app loads your existing account instead of asking again.

The users table holds your account and renter info, plus other users' data and (in the second project) passwords. One row looks like:

```
id          1  
username    liisa                # unique  
password    NULL                # empty in this project  
created_at  2026-06-16 14:30          # set by the database
```

You design the table from this.

On start-up the app opens the database file, creating it and the tables if they don't exist yet.

Listing items

Listing an item prompts for three fields in order: name, description, and cost per day, then returns to the main menu once the item is saved.

Each item you list is stored in `listed_items`. One row looks like:

```
item_id      1  
owner_id     1                # the user who listed it  
item_name    Ladder  
description   6-step aluminium ladder  
cost_per_day 5
```

```
status      available      # only available, rented, or unlisted (database)
created_at  2026-06-16 14:30      # set by the database
```

Viewing and delisting items

"View my inventory" lists every item the user owns, each row showing just its name and status.

Every list in the app works the same way: numbered and paginated past 5 items, with next, previous, and a back-to-main-menu option. Next and previous appear only when those pages exist. Picking a numbered entry opens its detail view, where you carry out that screen's action (delist, rent, or confirm a return, depending on the list).

For example, the inventory list:

```
=== My inventory ===
1) Ladder      available
2) Drill       rented
3) Tent        unlisted
4) Sander      available
5) Chainsaw    available
6) Next page
7) Back to menu
>
```

The examples use numbers, but the controls and how the user picks them are up to you.

An item's detail view shows its name, description, cost per day, status, the owner's username, and when it was listed:

```
=== Ladder ===
description    6-step aluminium ladder
cost per day   5
status        available
owner         liisa
listed        2026-06-16 14:30
1) Delist
2) Back to list
>
```

From the detail view you can delist an item, setting its status to `unlisted`. An unlisted item can't be rented and isn't shown in the record a rental list, but it still appears in your inventory as `unlisted`. You can relist it back to `available`, unless it's still out on a rental, in which case it has to be returned first.

Implement the item's status as a **state machine**: a fixed set of states with only certain moves allowed between them. Only the transitions below are allowed, and your state machine enforces them in code, not just in the database:



Renting and returning

"Record a rental" shows the user's available items by name and cost per day. From an item's detail, the user records the rental by entering the renter's name and the duration in days. A renter who isn't in the users table yet is created from their name, and one who already exists is reused rather than duplicated. The item's status flips to rented, so it drops off this list and can't be rented again until it's returned.

"Confirm a return" lists the items the user currently has out by item name and renter, sorted by due date. The rental detail view shows the item name, the renter's username, the start and end times, and the status:

```
=== Ladder ===  
renter    meelis  
start     2026-06-16 14:30  
end       2026-06-19 14:30  
status    active  
1) Confirm return  
2) Back to list  
>
```

From there, confirming the return marks the rental closed, writes the return timestamp to its record, and sends the item back to available. If the item was delisted while out, it stays unlisted instead. The rental record is never deleted. It's the history of who had what and when.

Each rental is stored in rentals. One row looks like:

```
rental_id    1  
item_id      1          # the rented item
```

```
renter_id    2                # the renter
start_time   2026-06-16 14:30 # when the rental begins
end_time     2026-06-19 14:30 # start_time plus the rental duration
returned_at  NULL              # empty until returned
status       active           # active while out, closed once returned (data
```

Handling database failures

Every database call in the repository can fail. Catch each failure in the repository and throw a **typed exception** that names which kind it is, so the service layer can decide what to do and the app can tell the user and return to the menu, instead of crashing or carrying on with a wrong result.

Your app must handle these exceptions:

- **can't open the database** - the file won't open (missing, wrong path, or bad permissions)
- **constraint violation** - a write breaks a NOT NULL, UNIQUE, CHECK, or foreign key rule
- **not found** - a query expects a row and finds none
- **mapping exception** - a row won't map into its domain object (wrong type or an unexpected NULL)

User flow

The whole app runs as one menu loop:



Testing

You can use **sample-data.sql** during development to populate your DB and check that core functionalities work.

For deliverables you have to build an automated test suite that proves the app works. Tests run against a real SQLite instance, a temporary file wiped after each run, never a mock. Each test sets up the state it needs, runs one operation, then checks the database directly instead of trusting the method's return value. Check test coverage with **JaCoCo**, cover 100% of your code, and commit the coverage report. Coverage points out code no test exercises, but reaching 100% only proves every line ran, not that the tests are any good.

Three groups of tests:

Data layer: CRUD exercised straight against the repository (no delete, items are soft-removed via status).

- insert users, items, and rentals
- query by ID and by filter, including incorrect inputs that fail correctly without crashing
- update item and rental status

Constraints: also against the repository, where violations surface. Each violation must throw the specific exception for the rule it broke, and the test checks for that exact exception, not just that the

write failed.

- a duplicate username (UNIQUE)
- a rental pointing to a non-existent item (foreign key)
- an item missing a required field (NOT NULL)
- an item with an out-of-range status (CHECK)

Behaviour: these tests go through the service layer, where the state machine and business rules are enforced, not in the repository.

- *State transitions:*
 - first launch creates the user, and a second run loads that same account instead of creating a new one
 - listing sets an item to available
 - recording a rental sets item to rented
 - confirming a return sets item back to available, unless it was delisted while out, in which case it stays unlisted
- *Business rules:*
 - an already-rented item can't be rented again
 - a delisted item can't be selected for rental
 - renting to an existing username reuses that record instead of creating a new one
 - end time equals the start time plus the number of days entered

What "tested enough" looks like:

- Every case in the three groups above has a test.
- The normal cases and the edge cases named above are covered, not just the happy path.
- Tests are isolated: each builds its own data directly through the repository, and they can pass in any order.
- All tests pass.

Deliverables and review requirements

Your git repository has to contain:

- the working CLI application with the required tests passing
- a test coverage report for the test suite
- a DB data file seeded with at least 6 listed items
- an ER diagram of all three tables, showing their columns, types, constraints, and relationships
- git history that shows the work as a team: branches merged via reviewed PRs, with small, focused commits and clear messages rather than one large bulk commit

Be ready to demonstrate the full lifecycle:

- first launch through to the main menu
- listing an item
- viewing inventory and delisting
- recording a rental with a new renter
- confirming a return
- running the test suite

Be ready to explain:

- how the layered architecture is enforced in the codebase
- how database exceptions are caught and handled at the repository layer